Chapter 2: Elementary Programming

COMP 110
Examples, discussion, and practice

Learning goals

Turn a problem statement into an algorithm
Declare variables and named constants
Read numeric input with Scanner
Evaluate expressions and numeric conversions
Trace assignments and diagnose common errors
Test calculations against known results

From a problem to a tested program

Problem: find the area of a circle with radius 20.

Input: radius, measured in a stated unit
Output: area, in square units
Process: area = radius × radius × pi
Check: doubling the radius should quadruple the area.

An algorithm before Java syntax

1. Set the radius to 20.
2. Multiply radius by itself and by pi.
3. Display the area.

Discussion: What changes when the user supplies the radius?

Ways to organize program execution

Structure	Purpose and example
Sequence	Run statements in order: read, calculate, print.
Selection	Choose a path: accept or reject an input.
Iteration	Repeat a task: process several values.
Methods	Name a task so it can be called and reused.

This chapter focuses on sequences of input, calculation, and output.

The Java application skeleton

```
public class SimpleProgram {
    public static void main(String[] args) {
        System.out.println("Hello, COMP 110!");
    }
}
```

Save as SimpleProgram.java. The main method is the entry point.

Java: SimpleProgram.java

Circle area: a complete calculation

```
double radius = 20;
final double PI = 3.14159;
double area = radius * radius * PI;
System.out.println("Area = " + area);
```

The right side is evaluated before its result is stored in area.

Java: ComputeArea.java

Discussion: what should a circle test prove?

Predict the area for radii 0, 1, and 2.
Compare radius 20 with radius 40.

Would a successful compilation catch using
2 × radius × pi instead of radius × radius × pi?

Identifiers and readable names

Use names such as radius, hourlyRate, and totalPay.
Start with a letter in ordinary course code.
Names are case-sensitive: rate and Rate differ.
Do not use a keyword or true, false, or null.

Class: GrossPay Variable: hourlyRate
Constant: TAX_RATE

Names and declarations

```
int studentCount = 24;
double hourlyRate = 18.5;
char grade = 'A';
// int 2students = 24;   // cannot start with a digit
// int class = 1;        // reserved keyword
// double hourly-rate;   // hyphen is not part of a name
System.out.println(studentCount);
System.out.println(hourlyRate);
System.out.println(grade);
```

Invalid declarations stay commented so the example compiles.

Java: IdentifierExamples.java

Declaration and initialization

```
int count;
count = 3;
double radius = 1.4;
char section = 'A';
System.out.println(count);
System.out.println(radius);
System.out.println(section);
```

A local variable must receive a value before it is read.

Java: DeclareAndInitialize.java

Assignment changes the stored value

```
int x = 10;
int y = x + 1;
x = 20;
x = x + 2;
System.out.println("x = " + x);
System.out.println("y = " + y);
```

Predict both outputs before running the file.

Java: AssignmentTrace.java

Tracing the assignments

After this statement	Stored values
int x = 10;	x: 10
int y = x + 1;	x: 10 y: 11
x = 20;	x: 20 y: 11
x = x + 2;	x: 22 y: 11

Only the variable on the left receives a new value.

Named constants express intent

```
final double PI = 3.14159;
final int MAX_STUDENTS = 30;
double radius = 2;
double area = PI * radius * radius;
System.out.println(area);
System.out.println(MAX_STUDENTS);
// PI = 3.14; // cannot reassign this final variable
```

final prevents reassignment. Uppercase names are a convention.

Java: NamedConstants.java

Quiz 1: variables and names

1. Are total and Total the same variable?
2. What is y after: x = 2; y = x; x = 9; ?
3. Can a final variable be assigned a new value?
4. May a local variable be printed before initialization?
5. Is hourlyRate a class-name convention?

Java: VariablesQuiz.java

Primitive numeric types

Type	Value model
byte / short	8-bit / 16-bit signed integers
int / long	32-bit / 64-bit signed integers
float / double	32-bit / 64-bit binary floating-point values
char	16-bit unsigned UTF-16 code unit

Use int for ordinary counts and double for these measured quantities.

Choosing a type and a literal

```
byte smallCount = 100;
short seats = 300;
int population = 1_000_000;
long distance = 3_000_000_000L;
float temperature = 18.5F;
double radius = 18.5;
System.out.println(smallCount + " " + seats);
System.out.println(population + " " + distance);
System.out.println(temperature + " " + radius);
```

L marks a long literal. F marks a float literal.

Java: NumericTypes.java

Number literals are values in the source

```
int count = 1_000_000;
long total = 4_000_000_000L;
double scale = 1.25e3;
double fraction = 5.0e-2;
System.out.println(count);
System.out.println(total);
System.out.println(scale);
System.out.println(fraction);
```

Write 1_000_000 in Java, rather than using grouping commas.

Java: NumberLiterals.java

Discussion: the type follows the quantity

Choose a type for each quantity and explain why.

Number of students in a section
Measured radius of a circle
A national population above three billion
An exact amount of money in whole cents

Reading numeric input with Scanner

```
Scanner input = new Scanner(System.in);
input.useLocale(Locale.US);
System.out.print("Enter an integer and a decimal: ");
int count = input.nextInt();
double rate = input.nextDouble();
System.out.println("Count = " + count);
System.out.println("Rate = " + rate);
```

Imports at the top of the file make Scanner and Locale available.

Java: ReadNumbers.java

Circle area with keyboard input

```
Scanner input = new Scanner(System.in);
input.useLocale(Locale.US);
System.out.print("Enter radius: ");
double radius = input.nextDouble();
final double PI = 3.14159;
double area = radius * radius * PI;
System.out.println("Area = " + area);
```

The calculation stays the same when the source of radius changes.

Java: ComputeAreaWithInput.java

Gross pay: define the model first

Input: hours worked and hourly pay rate
Output: gross pay before deductions
Formula: grossPay = hours × hourlyRate

For 8 hours at $18.50 per hour, expect $148.00.
Discussion: Does this model include overtime?

Gross pay: input, process, output

```
Scanner input = new Scanner(System.in);
input.useLocale(Locale.US);
System.out.print("Enter hours and hourly rate: ");
double hours = input.nextDouble();
double hourlyRate = input.nextDouble();
double grossPay = hours * hourlyRate;
System.out.printf(Locale.US, "Gross pay: $%.2f%n", grossPay);
```

%.2f displays two digits after the decimal point.

Java: GrossPay.java

Quiz 2: input and numeric types

1. Which Scanner method reads an integer token?
2. Which type fits a radius such as 2.75?
3. What does the L in 4_000_000_000L mean?
4. What is wrong with writing 1,000 as a Java number?
5. Which gross-pay inputs should produce zero?

Arithmetic operators

Operator	Meaning and example
+ and -	Addition and subtraction: 9 + 4 is 13.
*	Multiplication: 9 * 4 is 36.
/	Division: 9 / 4 is 2; 9 / 4.0 is 2.25.
%	Remainder: 9 % 4 is 1.

Operand types affect the result of an operation.

Five operators, two integer operands

```
int a = 9;
int b = 4;
System.out.println(a + b);
System.out.println(a - b);
System.out.println(a * b);
System.out.println(a / b);
System.out.println(a % b);
```

Predict 13, 5, 36, 2, and 1, in that order.

Java: ArithmeticOperators.java

Integer division happens before assignment

```
System.out.println(5 / 2);
System.out.println(5 / 2.0);
System.out.println(-5 / 2);
double first = 5 / 2;
double second = 5.0 / 2;
System.out.println(first);
System.out.println(second);
```

A double destination cannot restore a fraction already discarded.

Java: IntegerDivision.java

Remainder separates whole units

```
int totalSeconds = 3672;
int hours = totalSeconds / 3600;
int remaining = totalSeconds % 3600;
int minutes = remaining / 60;
int seconds = remaining % 60;
System.out.println(hours + " h " + minutes + " m " + seconds + " s");
System.out.println(17 % 5);
```

3672 seconds contains 1 hour, 1 minute, and 12 seconds.

Java: RemainderAndTime.java

Precedence and grouping change results

```
System.out.println(2 + 3 * 4);
System.out.println((2 + 3) * 4);
System.out.println(20 / 5 * 2);
System.out.println(20 / (5 * 2));
System.out.println("Sum = " + 2 + 3);
System.out.println("Sum = " + (2 + 3));
```

Parenthesize arithmetic inside a text output expression.

Java: OperatorPrecedence.java

Translating a mathematical expression

```
double x = 2, y = 4;
double a = 1, b = 2, c = 3;
double result = (3 + 4 * x) / 5
    - 10 * (y - 5) * (a + b + c) / x
    + 9 * (4 / x + (9 + x) / y);
System.out.println(result);
```

Use explicit multiplication and parentheses around every numerator.

Java: ArithmeticExpression.java

Quiz 3: evaluate before running

1. 7 / 2
2. 7 / 2.0
3. 17 % 5
4. 2 + 3 * 4
5. (2 + 3) * 4
6. double average = (4 + 5) / 2;

Java: ArithmeticQuiz.java

Converting Fahrenheit to Celsius

```
Scanner input = new Scanner(System.in);
input.useLocale(Locale.US);
System.out.print("Enter Fahrenheit: ");
double fahrenheit = input.nextDouble();
double celsius = (5.0 / 9) * (fahrenheit - 32);
System.out.printf(Locale.US, "Celsius: %.2f%n", celsius);
```

Check 32°F → 0°C, 212°F → 100°C, and -40°F → -40°C.

Java: FahrenheitToCelsius.java

Floating-point values have finite precision

```
System.out.println(1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1);
System.out.println(1.0 - 0.9);
System.out.println(0.5 + 0.25);
System.out.printf(Locale.US, "Display: %.2f%n", 1.0 - 0.9);
```

Some fractions are exact in binary. Many decimal fractions are not.

Java: FloatingPointApproximation.java

Integers are exact within a finite range

```
int largest = 2_147_483_647;
int wrapped = largest + 1;
long widenedTooLate = largest + 1;
long safe = (long) largest + 1;
System.out.println(wrapped);
System.out.println(widenedTooLate);
System.out.println(safe);
```

Widen an operand before arithmetic when the int result would overflow.

Java: IntegerOverflow.java

Discussion: classify the failure

A misspelled variable prevents compilation.
An integer division by zero stops execution.
A circle program uses circumference as its area.
A large int addition produces a negative value.

Which check would expose each problem?

Compound assignment updates one variable

```
int count = 10;
count += 3;
System.out.println(count);
count -= 2;
System.out.println(count);
count *= 4;
System.out.println(count);
count /= 5;
System.out.println(count);
count %= 3;
System.out.println(count);
```

For these int operands, count += 3 behaves like count = count + 3.

Java: CompoundAssignment.java

Increment and decrement as statements

```
int count = 5;
count++;
System.out.println(count);
++count;
System.out.println(count);
count--;
System.out.println(count);
--count;
System.out.println(count);
```

As standalone statements, ++count and count++ both add one.

Java: IncrementDecrement.java

Prefix and postfix return different values

```
int first = 5;
int savedFirst = first++;
System.out.println(first + " " + savedFirst);
int second = 5;
int savedSecond = ++second;
System.out.println(second + " " + savedSecond);
```

Postfix uses the old value; prefix uses the updated value.

Java: PrefixPostfix.java

Numeric promotion follows operand types

```
byte i = 100;
long k = i * 3 + 4;
double d = i * 3.1 + k / 2;
System.out.println(k);
System.out.println(d);
System.out.println(k / 3);
System.out.println(k / 3.0);
```

Evaluate each operation using its own operand types.

Java: NumericPromotion.java

Binary numeric promotion

Operand types	Type used for this numeric operation
Either is double	double
Otherwise, either is float	float
Otherwise, either is long	long
Otherwise	int, including byte / short / char operands

These rules apply per operation, before assignment to a destination.

Casting controls a conversion

```
double widened = 3;
int positive = (int) 3.9;
int negative = (int) -3.9;
double late = (double) (5 / 2);
double early = (double) 5 / 2;
System.out.println(widened);
System.out.println(positive + " " + negative);
System.out.println(late + " " + early);
// int x = 5 / 2.0; // explicit narrowing is required
```

For these finite values, double-to-int casting truncates toward zero.

Java: TypeCasting.java

Compound assignment includes a conversion

```
int sum = 0;
sum += 4.5;
System.out.println(sum);
sum += 4.5;
System.out.println(sum);
int explicit = 0;
explicit = (int) (explicit + 4.5);
System.out.println(explicit);
// sum = sum + 4.5; // double cannot be assigned to int
```

sum += 4.5 narrows the result back to the type of sum.

Java: CompoundAssignmentCasting.java

Quiz 4: trace state and conversions

1. int x = 4; int y = x++; What are x and y?
2. int a = 4; int b = ++a; What are a and b?
3. What is (int) -2.9?
4. What is (double) (7 / 2)?
5. int sum = 0; sum += 4.5; sum += 4.5;

Java: ConversionsQuiz.java

Two decimal places: display or value?

```
double value = 12.345;
double truncated = (int) (value * 100) / 100.0;
System.out.println("Stored: " + value);
System.out.println("Truncated: " + truncated);
System.out.printf(Locale.US, "Formatted: %.2f%n", value);
System.out.println("Still stored: " + value);
```

Formatting rounds the display. A cast-based expression truncates.

Java: DecimalDisplay.java

Sales tax with a named rate

```
Scanner input = new Scanner(System.in);
input.useLocale(Locale.US);
final double TAX_RATE = 0.06;
System.out.print("Enter purchase amount: ");
double purchase = input.nextDouble();
double tax = purchase * TAX_RATE;
System.out.printf(Locale.US, "Tax: $%.2f%n", tax);
System.out.printf(Locale.US, "Total: $%.2f%n", purchase + tax);
```

The sample rate is 6%, represented as 0.06.

Java: SalesTax.java

Discussion: a number has units

Is an entered rate of 6 the same as 0.06?
Does a monthly rate equal an annual rate?
Does showing $0.10 make a double exact?

Before coding, write down input units,
conversion steps, and expected output units.

Loan payments: state the assumptions

Inputs: annual percentage rate, years, principal
Convert percent per year to a monthly fraction.
Convert years to number of monthly payments.
Compute a fixed monthly payment.
Multiply by payment count for the unrounded total.

Loan payments: the calculation

```
double monthlyRate = annualRate / 1200;
int paymentCount = years * 12;
double monthlyPayment = principal * monthlyRate
    / (1 - Math.pow(1 + monthlyRate, -paymentCount));
double totalPayment = monthlyPayment * paymentCount;
```

Math.pow(base, exponent) computes a power. ^ is not exponentiation.

Java: LoanPayment.java

Loan payments: interpreting the result

```
System.out.printf(Locale.US, "Monthly: $%.2f%n", monthlyPayment);
System.out.printf(Locale.US, "Total: $%.2f%n", totalPayment);
```

Sample input 5 30 100000 gives $536.82 monthly and $193,255.78 total.

Java: LoanPayment.java

Monetary units: work with whole cents

For this exercise, enter an exact count of cents.
Split into dollars, quarters, dimes, nickels, pennies.
Use division for each whole-unit count.
Use remainder to keep the cents still unallocated.

Example: 1156 cents represents $11.56.

Monetary units: division and remainder

```
int dollars = remaining / 100;
remaining %= 100;
int quarters = remaining / 25;
remaining %= 25;
int dimes = remaining / 10;
remaining %= 10;
int nickels = remaining / 5;
int pennies = remaining % 5;
```

Each remainder becomes the input to the next smaller unit.

Java: MonetaryUnits.java

Monetary units: a trace for 1156 cents

Unit	Count and amount left
Dollars (100 cents)	11 dollars, 56 cents left
Quarters (25 cents)	2 quarters, 6 cents left
Dimes (10 cents)	0 dimes, 6 cents left
Nickels / pennies	1 nickel, then 1 penny

Check: $11 \times 100 + 2 \times 25 + 0 \times 10 + 1 \times 5 + 1 = 1156$.

Optional decimal entry: define a rounding rule

```
Scanner input = new Scanner(System.in);
input.useLocale(Locale.US);
System.out.print("Enter dollars: ");
double dollars = input.nextDouble();
long cents = Math.round(dollars * 100);
System.out.println("Nearest whole cents: " + cents);
```

Math.round is preferable here to silently truncating with a cast.

Java: DollarsToCents.java

Compile-time errors: names and initialization

```
double interestRate = 0.05;
double interest = interestRate * 45;
int count;
count = 3;
System.out.println(interest);
System.out.println(count);
// double wrong = interestrate * 45; // undeclared name
// int neverSet;
// System.out.println(neverSet); // not initialized
```

Use the first compiler message to locate the cause.

Java: VariableErrors.java

A logic error: unintended integer division

```
int first = 4;
int second = 5;
double wrong = (first + second) / 2;
double corrected = (first + second) / 2.0;
System.out.println("Before repair: " + wrong);
System.out.println("After repair: " + corrected);
```

A known result, 4.5, distinguishes the two versions.

Java: AverageRepair.java

One Scanner for the console input stream

```
Scanner input = new Scanner(System.in);
input.useLocale(Locale.US);
System.out.print("Enter an integer: ");
int first = input.nextInt();
System.out.print("Enter a decimal: ");
double second = input.nextDouble();
System.out.println("Values: " + first + " " + second);
```

Reuse input for both reads. Avoid competing buffers on System.in.

Java: SingleScanner.java

Quiz 5: diagnose and repair

1. Why can double x = 5 / 2; produce 2.0?
2. Does widening after int overflow repair the value?
3. Does printf with %.2f change the stored double?
4. Why test 99 and 100 cents?
5. Why use one Scanner for System.in?

Exit exercise: a three-score report

Read three scores as integers from 0 to 100.
Compute their total and their average.
Print the average with one decimal place.

Before running, predict results for 80, 81, 83.
Choose another test that catches integer division.

Exit exercise: one possible solution

```
Scanner input = new Scanner(System.in);
System.out.print("Enter three scores: ");
int first = input.nextInt();
int second = input.nextInt();
int third = input.nextInt();
int total = first + second + third;
double average = total / 3.0;
System.out.println("Total: " + total);
System.out.printf(Locale.US, "Average: %.1f%n", average);
```

The denominator 3.0 makes average a floating-point calculation.

Java: ThreeScoreReport.java

A small test plan

Program and input	Expected result
Circle: radius 0 and 2	Area 0.0 and 12.56636
Temperature: 32, 212, -40	Celsius 0.00, 100.00, -40.00
Money: 0, 99, 100 cents	No coins; 3Q + 2D + 4P; 1 dollar
Scores: 0 0 1	Total 1; displayed average 0.3

Use known values, boundary values, and a case that exposes a likely error.

Chapter review

Describe inputs, outputs, units, and assumptions.
Trace stored values one statement at a time.
Choose operand types before doing arithmetic.
Use parentheses to show the intended formula.
Separate numeric calculation from display formatting.
Test a known answer and a useful boundary.